STRIPE PROJECT

3. NoSQL Data System

Annexes

3.A. NoSQL Storage Model Rationale

> *Nicolas Pichon --- AIA RNCP 38777 / BC02 / NoSQL Data System / Annexe A / v09 --- 2026/11/05.*

- 3.A.1. Pourquoi choisir une base orientée _document_?
- 3.A.2. Sources de données
- 3.A.3. Discussion sur les autres familles de stockage
- 3.A.4. Avantages spécifiques du stockage par documents
  - 3.A.4.1. Le format correspond déjà à la donnée source
  - 3.A.4.2. Schéma flexible sans migration
  - 3.A.4.3. Requêtage riche sur des champs imbriqués
  - 3.A.4.4. Capture des changements par _Change Streams_
  - 3.A.4.5. _Sharding_ horizontal natif
- Références

3.A.1. Pourquoi choisir une base orientée *document*?

La première exigence technique du cahier des charges (TR1 dans [D0]) demande explicitement que le système soit capable de requêter efficacement des données non structurées imbriquées (*"efficient querying of nested and unstructured data"*). Cette exigence élimine, de fait, les trois autres familles de stockage *NoSQL* (cf. analyse ci-dessous).

3.A.2. Sources de données

D'après [D0], les sources de données que *NoSQL* doit prendre en compte sont les suivantes (cf. [D0, §0.2.6/DS3]) :

- *Clickstreams des utilisateurs & données de sessions* : un événement de clic n'a pas les mêmes attributs qu'une soumission de formulaire ou qu'une vue de page ; la structure varie par nature d'un enregistrement à l'autre.
- *Machine Learning features (détection de fraude)* : un vecteur de caractéristiques évolue au fil des itérations du modèle : on ajoute, retire, recombine des variables sans prévenir.
- *Retours clients (avis, enquêtes)* : texte libre plus métadonnées dont les champs varient selon le canal de collecte.

Point commun de ces sources : une structure imprévisible et changeante, et de gros volumes.

3.A.3. Discussion sur les autres familles de stockage

Stockage par clé-valeur (*Redis*, *DynamoDB*)

Excellent pour une lecture par clé unique, mais aucune capacité native à interroger *à l'intérieur* de la donnée. Impossible de requêter "*tous les événements t.q.* `device_type = mobile` *et* `event_type = click`" sans tout rapatrier du côté de l'application. Or c'est exactement ce type de requête qu'exige l'exigence TR1 [D0].

Stockage par colonne (*Cassandra*, *HBase*)

Très bon pour un débit d'écriture extrême sur un schéma de colonnes relativement stable, mais une famille de colonnes reste rigide par nature. Mal adapté à des enregistrements dont la forme change d'un type d'événement à l'autre.

Stockage en graphe (*Neo4j*)

Pertinent pour des requêtes de traversée de relations (détection de réseaux de fraude coordonnée, par exemple), mais ce n'est pas le besoin ici : on stocke des événements et des vecteurs de caractéristiques, pas un graphe de relations à parcourir.

3.A.4. Avantages spécifiques du stockage par documents

3.A.4.1. Le format correspond déjà à la donnée source

Un événement de *clickstream* envoyé par une app web/mobile est nativement un objet JSON. Le stocker tel quel, sans le forcer dans un schéma relationnel fixe, évite une transformation qui n'apporterait rien.

3.A.4.2. Schéma flexible sans migration

Un document non structuré de type JSON absorbe immédiatement les nouveaux types d'événement ou les nouveaux champs de feature, sans restructuration de tables SQL ni migration des enregistrements existants. Cette flexibilité permet au modèle de données d'évoluer continûement (nouvelles features testées, nouvelle instrumentation produit) sans surcoût significatif.

3.A.4.3. Requêtage riche sur des champs imbriqués

Pour répondre à l'exigence technique "*Efficient querying of nested and unstructured data*" (TR1 [D0]), *MongoDB* propose des mécanismes optimisés d'accès aux données imbriqués (et dont on ne disposera pas dans les systèmes de stockage par clé-valeur comme *Redis* ou *DynamoDB*) :

- Indexation secondaire sur des sous-champs ;
  - Par exemple, ce type d'indexation permet de trier ou de filtrer directement sur le champ `fraud_v3` qui est lui-même imbriqué dans la structure `model_scores` de la collection `ml_features` (cf. *Collections* dans [D4]) ;
- Indexation *multikey* sur des tableaux imbriqués ;
  - Exemple : dans la collection `user_sessions`, `events` est un tableau imbriqué dans chaque objet de session, `metadata` est une structure imbriquée dans chaque objet d'évènement; le champ `metadata.transaction_id` est donc niché à deux niveaux à l'intérieur de chaque élément de ce tableau (cf. *Collections* dans [D4]).
  - *MongoDB* crée automatiquement une entrée d'index par élément du tableau, de sorte qu'une requête comme `findOne({"events.metadata.transaction_id": "b7c2..."})` retrouve directement la session concernée sans parcourir un seul document ligne par ligne.

3.A.4.4. Capture des changements par *Change Streams*

MongoDB dispose nativement d'un mécanisme de capture des modifications basé sur des flux d'évènements continus dits *Change Streams*. Ecouter ces flux en temps réel permet au système d'alimenter le pipeline *NoSQL -> OLAP* via l'orchestration *Airflow* (cf. architecture globale [D1]).

3.A.4.5. *Sharding* horizontal natif

Pour répondre à l'exigence technique "*distributed databases and data sharding*" (TR3 [D0]), *MongoDB* propose nativement des capacités de *sharding* horizontal. Cette technique de partitionnement permet de mettre le système à l'échelle horizontalement (*horizontal scaling*) sans avoir besoin de revoir le modèle.

Références

Spécifications

- [D0] Stripe Project Introduction
-

Livrables

- [D1] Data Architecture Diagram
- [D4] NoSQL Data Model